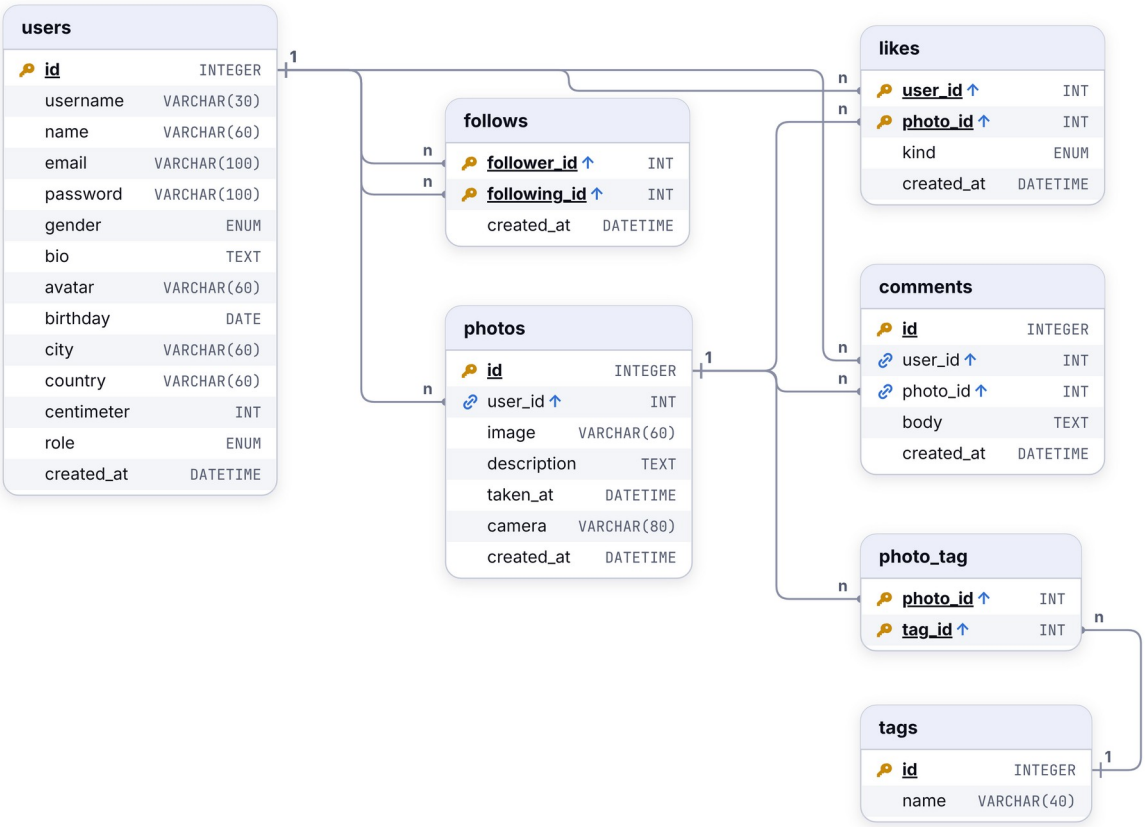
INFORMATIK · QUALIFIKATIONSPHASE

DATENBANK SYSTEME

Lösungen und Hinweise für die Lehrkraft



Sönke Birkner
soenke.birkner@mps-ki.de

Hinweise zum Einsatz

- **Reihenfolge der Tabellen** (Startseite der Lehrkraft → Tabellen für alle Netzwerke → Erstellen, danach „Füllen“): II.1 nur **users** · II.2 **photos** und **comments** · II.3 **tags** (bringt photo_tag mit), **follows** und **likes** · II.7 **sessions** und **views** · II.8 **ads**.
- **Daten neu füllen vor Kapitel 7:** In Kapitel 6 löschen die Schülerinnen und Schüler u. a. alle Minderjährigen (tim.richter, ida.hsn, lasse.ld …) und ändern Likes, Fotos und Passwörter. Vor Analytics deshalb **users** → **Leeren** und danach alles wieder **Füllen** (Leeren nimmt die abhängigen Tabellen mit).
- **Ergebnisse** wurden mit den Dummy-Daten gerechnet (Netzwerk „Liky“, Stichtag 2026-11-07). Alle Zeiten (Alter, Registrierung, Posts) wachsen mit dem Raum mit – Aussagen wie „Erna ist 78“ oder „Noah wird morgen 18“ gelten an jedem Stichtag. Absolute Datumswerte unterscheiden sich.
- **Eigene Zeilen der Klasse** verändern einige Ergebnisse: Das eigene Admin-Konto ist die neueste Registrierung und hat die Rolle admin; eigene Posts sind die neuesten Fotos. Die eigene id ist in den Lösungen **71**.
- **E-Mail-Domain** = Name des Netzwerks, klein und ohne Sonderzeichen (hier `@liky.test`).
- **Klassen-Netz:** Für Aufgabe 12 und 13 c) brauchen die Gruppen dort Admin-Rechte: `UPDATE users SET role = 'admin' WHERE username IN (…)`. Im Klassen-Netz gibt es keine Dummy-User, aber die Beispiel-Werbung.
- **Fehlermeldungen sind manchmal gewollt** (7 l, 10 a, 10 d): Sie sind in den Lösungen rot markiert.
- ***** an einer Teilaufgabe = nur mit den Befehlen aus Abschnitt 4 (Tiefer in SQL) lösbar; Stern = zum Knobeln (freiwillig, schwerer).

Aufgabe 1

a) Gib alle Einträge aus der Tabelle users aus.

```
SELECT *
FROM users
```

Ergebnis: 70 Zeilen (Ausschnitt)

id	username	name	gender	birthday	city
1	pixelpaul	Paul Brandt	male	2007-06-20	Leipzig
2	noobqueen	Mia Sander	female	2009-09-16	Kiel
3	hugo.ranked	Hugo Wendt	male	2011-01-10	Berlin

b) Gib alle Benutzernamen (username) und echten Namen (name) aller Einträge aus users aus.

```
SELECT username, name
FROM users
```

Ergebnis: 70 Zeilen (Ausschnitt)

username	name
pixelpaul	Paul Brandt
noobqueen	Mia Sander
hugo.ranked	Hugo Wendt

c) Gib die Wohnorte aller Mitglieder aus.

```
SELECT username, city
FROM users
```

Ergebnis: 70 Zeilen (Ausschnitt)

username	city
pixelpaul	Leipzig
noobqueen	Kiel
hugo.ranked	Berlin

d) Gib die Größe der User in Fuß (feet) an (1 cm = 0,0328 ft).

```
SELECT username, centimeter * 0.0328 AS feet
FROM users
```

Ergebnis: 70 Zeilen (Ausschnitt)

username	feet
pixelpaul	5.97
noobqueen	5.41
hugo.ranked	5.61

> **HINWEIS**
>
> Achtung: Dezimalpunkt statt Komma. User ohne Größe (NULL) bleiben leer.

e) Berechne das Alter der User und zeige den Namen und das Alter an. Nutze die Funktionen von der vorherigen Seite.

```
SELECT name, FLOOR(DATEDIFF(NOW(), birthday) / 365.25) AS jahre
FROM users
```

Ergebnis: 70 Zeilen (Ausschnitt)

name	jahre
Paul Brandt	19
Mia Sander	17
Hugo Wendt	15

> **HINWEIS**
>
> Ohne FLOOR entstehen Kommazahlen. Wer durch 365 teilt, merkt es in f).

f) Überprüfe mit deiner Lösung das Alter von Noah (Username noah.sz). Stimmt dein Ergebnis? Falls nicht, überlege dir, woran das liegen könnte.

BEFEHL 1

```sql
SELECT name, FLOOR(DATEDIFF(NOW(), birthday) / 365) AS jahre
FROM users
WHERE username = 'noah.sz'
```

Ergebnis: 1 Zeile

name	jahre
Noah Schulz	18

BEFEHL 2

```sql
SELECT name, FLOOR(DATEDIFF(NOW(), birthday) / 365.25) AS jahre
FROM users
WHERE username = 'noah.sz'
```

Ergebnis: 1 Zeile

name	jahre
Noah Schulz	17

> **HINWEIS**
>
> Mit /365 kommt schon 18 heraus: In 18 Jahren liegen 4–5 Schalttage, 17 Jahre und 364 Tage sind also mehr als 18 · 365 Tage. Mit /365.25 (ein Jahr im Durchschnitt) stimmt es: 17.

Aufgabe 2

a) Gib alle Wohnorte in einer sortierten Liste aus, bei der jede Stadt einmalig auftritt.

```sql
SELECT DISTINCT city
FROM users
ORDER BY city
```

Ergebnis: 34 Zeilen (Ausschnitt)

city
NULL
Aachen
Aarhus

> **HINWEIS**
>
> Hier fallen schon „Hamburg“, „hamburg“, „Hambrug“ und eine leere Zeile (NULL) auf.

b) Ordne die Mitglieder nach ihrer Größe absteigend.

```
SELECT name, centimeter
FROM users
ORDER BY centimeter DESC
```

Ergebnis: 70 Zeilen (Ausschnitt)

name	centimeter
Ronny Pfeiffer	250
Mikkel Sørensen	193
Jannik Wolf	191

> **HINWEIS**
>
> Ganz oben: Ronny Pfeiffer mit 250 cm – eine Falschangabe des Trolls. Gesprächsanlass: Wie verlässlich sind selbst angegebene Daten?

c) Kehre die Reihenfolge der Tabelle **users** um.

```
SELECT *
FROM users
ORDER BY id DESC
```

Ergebnis: 70 Zeilen (Ausschnitt)

id	username	name	gender	birthday	city
70	geloescht_4711	Gelöschter Nutzer	*NULL*	*NULL*	*NULL*
69	jule.w	Jule Winter	female	2007-10-28	Kiel
68	test_account	*NULL*	*NULL*	*NULL*	*NULL*

d) Gib die fünf zuletzt registrierten User aus.

```
SELECT username, created_at
FROM users
ORDER BY created_at DESC
LIMIT 5
```

Ergebnis: 5 Zeilen (Ausschnitt)

username	created_at
jule.w	2026-11-05 17:13:45
gewinn_jetzt_2026	2026-09-28 19:32:18
lotta.b	2026-09-08 16:31:21

> **HINWEIS**
>
> Auf Platz 1 steht jule.w – außer jemand aus der Klasse hat sich danach registriert (das eigene Admin-Konto!).

Aufgabe 3

a) Teste beide Möglichkeiten vom letzten Abschnitt und finde heraus, worin der Unterschied der Ausgaben liegt. Welche Möglichkeit passt zur Aufgabe?

BEFEHL 1

```
SELECT name, city, gender
FROM users
WHERE city = 'Kiel' AND gender = 'male' OR gender = 'female'
```

Ergebnis: 33 Zeilen (Ausschnitt)

name	city	gender
Mia Sander	Kiel	female
Levi Schumacher	Kiel	male
Fee Lorenz	München	female

BEFEHL 2

```
SELECT name, city, gender
FROM users
WHERE city = 'Kiel' AND (gender = 'male' OR gender = 'female')
```

Ergebnis: 11 Zeilen (Ausschnitt)

name	city	gender
Mia Sander	Kiel	female
Levi Schumacher	Kiel	male
Ralf Opitz	Kiel	male

HINWEIS

Möglichkeit 1 liefert Kieler Männer und ALLE Frauen (AND vor OR). Möglichkeit 2 ist richtig.

b) Gib alle User aus, die nicht aus Deutschland kommen. Schau dir dein Ergebnis genau an: Stimmt es wirklich?

```
SELECT username, name, country
FROM users
WHERE country != 'Deutschland'
```

Ergebnis: 10 Zeilen (Ausschnitt)

username	name	country
fitbyfee	Fee Lorenz	Germany
vanlife_tomasz	Tomasz Kowalski	Polen
marco_hat_recht	Uwe Fink	deutschland

HINWEIS

Falle: „Germany“, „deutschland“ und „DE“ tauchen auf, User mit NULL fehlen ganz. Gelöst wird das in Aufgabe 7 a) mit IN.

c) Zeige nur Levi Schumacher an. Wie viele Treffer bekommst du? Wie kannst du sicherstellen, dass du genau einen bestimmten Levi bekommst?

```
SELECT *
FROM users
WHERE name = 'Levi Schumacher'
```

Ergebnis: 2 Zeilen (Ausschnitt)

id	username	name	gender	birthday	city
4	levi.kicks	Levi Schumacher	male	2009-04-11	Kiel
37	levi.berlin	Levi Schumacher	male	1992-05-03	Berlin

> **HINWEIS**
>
> Zwei Treffer (levi.kicks aus Kiel, levi.berlin aus Berlin). Eindeutig nur über den Primärschlüssel id (oder den eindeutigen username) – Rückbezug zu I.2.1.

d) Prüfe, ob es minderjährige User gibt.

```sql
SELECT name, FLOOR(DATEDIFF(NOW(), birthday) / 365.25) AS jahre
FROM users
WHERE FLOOR(DATEDIFF(NOW(), birthday) / 365.25) < 18
```

Ergebnis: 23 Zeilen (Ausschnitt)

name	jahre
Mia Sander	17
Hugo Wendt	15
Levi Schumacher	17

> **HINWEIS**
>
> Ja, 23 (mit /365 wären es nur 22 – siehe 1 f)).

e) ★ Finde zuerst heraus, wann levi.kicks geboren ist. Gib dann alle User aus, die im selben Jahr geboren sind, und ordne sie nach Geburtsdatum.

BEFEHL 1

```sql
SELECT birthday
FROM users
WHERE username = 'levi.kicks'
```

Ergebnis: 1 Zeile

birthday
2009-04-11

BEFEHL 2

```sql
SELECT name, birthday
FROM users
WHERE YEAR(birthday) = YEAR((SELECT birthday FROM users WHERE username = 'levi.kicks'))
ORDER BY birthday
```

Ergebnis: 6 Zeilen (Ausschnitt)

name	birthday
Leila Haddad	2009-01-01
Lars Peters	2009-01-31
Levi Schumacher	2009-04-11

> **HINWEIS**
>
> Schüler setzen das Jahr von Hand ein (z. B. `WHERE birthday LIKE '2009%'`).

f) Erstelle eine Liste mit Usern, deren E-Mail-Adresse zu deinem Netzwerk gehört (sie endet auf *@name-deines-netzwerks.test*).

```sql
SELECT username, email
FROM users
WHERE email LIKE '%@liky.test'
```

Ergebnis: 39 Zeilen (Ausschnitt)

username	email
pixelpaul	paul.brandt@liky.test
levi.kicks	levi.schumacher@liky.test
finn.kielwasser	finn@liky.test

> **HINWEIS**
>
> Der Name hängt vom Netzwerk ab (hier „Liky“). Die Domain steht klein und ohne Sonderzeichen.

g) Gib eine Liste aller männlichen User an, die größer als 1,85 m sind. Ordne die Liste nach Wohnort.

```sql
SELECT name, city, centimeter
FROM users
WHERE gender = 'male' AND centimeter > 185
ORDER BY city
```

Ergebnis: 8 Zeilen (Ausschnitt)

name	city	centimeter
Tomasz Kowalski	*NULL*	186
Mikkel Sørensen	Aarhus	193
Ronny Pfeiffer	Chemnitz	250

h) Prüfe, ob es einen User gibt, der Felix heißt und nicht aus Berlin kommt.

```sql
SELECT username, name, city
FROM users
WHERE name LIKE 'Felix%' AND city != 'Berlin'
```

Ergebnis: 1 Zeile

username	name	city
felix_ude	Felix Ude	Bremen

> **HINWEIS**
>
> Nur Felix Ude (Bremen). `name = 'Felix'` findet niemanden – im Attribut name steht der ganze Name.

i) ★ Die Bundeswehr sucht Rekrutinnen und Rekruten. Gib die Namen und das Alter aller Männer über 165 cm und aller Frauen über 160 cm aus (es werden keine Panzerfahrerinnen und Panzerfahrer gesucht).

```sql
SELECT name, gender, centimeter, FLOOR(DATEDIFF(NOW(), birthday) / 365.25) AS jahre
FROM users
WHERE (gender = 'male' AND centimeter > 165) OR (gender = 'female' AND centimeter > 160)
```

Ergebnis: 56 Zeilen (Ausschnitt)

name	gender	centimeter	jahre
Paul Brandt	male	182	19
Mia Sander	female	165	17
Hugo Wendt	male	171	15

> **HINWEIS**
>
> Klammern nötig. Wer will, filtert zusätzlich nach Alter ≥ 18.

j) Erna Lüders sucht ihren Enkel. Sein Vorname war entweder Lasse oder Lars. Er kommt wohl aus Berlin. Kannst du herausfinden, ob ihr Enkel in deinem Netzwerk ist?

```sql
SELECT username, name, city
FROM users
WHERE (name LIKE 'Lasse%' OR name LIKE 'Lars%') AND city = 'Berlin'
```

Ergebnis: 2 Zeilen

username	name	city
lasse.ld	Lasse Lüders	Berlin
larsole	Lars-Ole Kunz	Berlin

> **HINWEIS**
>
> Zwei Kandidaten: Lasse Lüders und Lars-Ole Kunz. Der Nachname verrät Lasse. Endgültig klärt es Aufgabe 6 g) über follows.

Aufgabe 4

a) Gib an, wann sich zuletzt ein Mitglied registriert hat.

```sql
SELECT MAX(created_at)
FROM users
```

Ergebnis: 1 Zeile

MAX(created_at)
2026-11-05 17:13:45

b) Gib das Alter des ältesten Mitglieds an.

```sql
SELECT MAX(FLOOR(DATEDIFF(NOW(), birthday) / 365.25)) AS jahre
FROM users
```

Ergebnis: 1 Zeile

jahre
78

> **HINWEIS**
>
> Erna Lüders, 78.

c) Betrachte die beiden Befehle und überlege, was diese ausgeben sollen. Teste deine Vermutung. Zu welchem Wert gehört der Name im zweiten Befehl? Erkläre, warum das Ergebnis anders ist als erwartet.

```sql
BEFEHL 1
SELECT name, MAX(centimeter)
FROM users
```

Ergebnis: 1 Zeile

name	MAX(centimeter)
Ronny Pfeiffer	250

BEFEHL 2

```
SELECT name, MAX(centimeter), MIN(centimeter)
FROM users
```

Ergebnis: 1 Zeile

name	MAX(centimeter)	MIN(centimeter)
Lotta Berg	250	150

HINWEIS

Befehl 1: Ronny Pfeiffer, 250. Befehl 2: Lotta Berg, 250, 150 – der Name gehört zum MIN, steht aber neben dem MAX. Eine Gruppe hat nur EINE Zeile; welcher Name dort steht, legt SQL fest, nicht die Funktion. Saubere Lösung mit Unterabfrage: Aufgabe 7 c).

d) Ermittle, wie viele männliche und wie viele weibliche User dein Netzwerk hat. Was steht außerdem in der Ausgabe?

```
SELECT gender, COUNT(*)
FROM users
GROUP BY gender
```

Ergebnis: 4 Zeilen (Ausschnitt)

gender	COUNT(*)
NULL	3
diverse	1
female	29

HINWEIS

Zusätzlich diverse und eine Gruppe NULL (keine Angabe).

e) ★ Prüfe, ob es User gibt, die am gleichen Tag Geburtstag haben. Wer ist es?

BEFEHL 1

```
SELECT birthday, COUNT(*) AS anzahl
FROM users
WHERE birthday IS NOT NULL
GROUP BY birthday
ORDER BY anzahl DESC
LIMIT 3
```

Ergebnis: 3 Zeilen

birthday	anzahl
2011-07-30	2
2014-04-16	1
2013-08-09	1

BEFEHL 2

```
SELECT name, birthday
FROM users
WHERE birthday = (SELECT birthday FROM users WHERE birthday IS NOT NULL GROUP BY birthday ORDER
BY COUNT(*) DESC LIMIT 1)
```

Ergebnis: 2 Zeilen

name	birthday
Jakob Huber	2011-07-30
Yusuf Demir	2011-07-30

> **HINWEIS**
>
> Jakob Huber und Yusuf Demir. Ohne `IS NOT NULL` steht die Gruppe NULL (3 User ohne Geburtstag) ganz oben. Falle: Mila Hoffmann und Elisa Brandt haben am selben Tag, aber in verschiedenen Jahren Geburtstag – zählt das als „gleicher Tag“? Diskussion.

f) Gib die Namen der drei Städte mit den meisten Usern an.

```sql
SELECT city, COUNT(*) AS anzahl
FROM users
GROUP BY city
ORDER BY anzahl DESC
LIMIT 3
```

Ergebnis: 3 Zeilen

city	anzahl
Kiel	11
München	7
Berlin	7

> **HINWEIS**
>
> Kiel 11, dann München und Berlin mit je 7.

g) Vergleiche die beiden Befehle. Warum kommen verschiedene Zahlen heraus? Finde die User, die im zweiten Befehl fehlen.

BEFEHL 1

```sql
SELECT COUNT(*), COUNT(centimeter)
FROM users
```

Ergebnis: 1 Zeile

COUNT(*)	COUNT(centimeter)
70	65

BEFEHL 2

```sql
SELECT username, centimeter
FROM users
WHERE centimeter IS NULL
```

Ergebnis: 5 Zeilen (Ausschnitt)

username	centimeter
gewinn_jetzt_2026	*NULL*
ben.fischer	*NULL*
max.mueller	*NULL*

> **HINWEIS**
>
> COUNT(*) zählt alle 70 Zeilen, COUNT(centimeter) nur die 65 mit Angabe. Auch AVG lässt NULL einfach weg.

h) Wie viele User kommen aus Hamburg? Vergleiche `city = 'Hamburg'` mit `city LIKE 'Hamburg%'`. Hast du wirklich alle Hamburger gefunden?

```
BEFEHL 1
SELECT COUNT(*)
FROM users
WHERE city = 'Hamburg'
```

Ergebnis: 1 Zeile

COUNT(*)
1

```
BEFEHL 2
SELECT city, COUNT(*)
FROM users
WHERE city LIKE 'Ham%'
GROUP BY city
```

Ergebnis: 4 Zeilen (Ausschnitt)

city	COUNT(*)
Hambrug	1
Hamburg	1
Hamburg	1

> **HINWEIS**
>
> = findet 1, LIKE 'Hamburg%' findet 3 (Hamburg, hamburg, „Hamburg “ mit Leerzeichen). „Hambrug“ (Tippfehler) findet nur ein genauer Blick. LIKE unterscheidet nicht zwischen Groß- und Kleinschreibung.

Aufgabe 5

a) Gib alle Fotos zusammen mit dem Benutzernamen des Besitzers aus.

```
SELECT users.username, photos.image, photos.description
FROM photos
JOIN users ON photos.user_id = users.id
```

Ergebnis: 151 Zeilen (Ausschnitt)

username	image	description
pixelpaul	dummy/paul_01.webp	Setup ist endlich fertig …
pixelpaul	dummy/paul_02.webp	Stream heute 20 Uhr!
pixelpaul	dummy/paul_03.webp	Platz 1 beim Leipziger Tu…

b) Gib eine absteigend sortierte Tabelle aus, in der die User und die Anzahl ihrer gepostеten Fotos stehen.

```
SELECT users.username, COUNT(*) AS fotos
FROM users
JOIN photos ON photos.user_id = users.id
GROUP BY users.id
ORDER BY fotos DESC
```

Ergebnis: 54 Zeilen (Ausschnitt)

username	fotos
alina.daily	7
wachtauf_marco	6
linus.lens	6

> **HINWEIS**
>
> Spitze: alina.daily mit 7. User ohne Foto fehlen hier (INNER JOIN) – das greift Aufgabe 7 g) auf.

c) Gib alle Fotos von Levi Schumacher aus Kiel mit ihrer Beschreibung aus.

```sql
SELECT photos.image, photos.description
FROM photos
JOIN users ON photos.user_id = users.id
WHERE users.username = 'levi.kicks'
```

Ergebnis: 6 Zeilen (Ausschnitt)

image	description
dummy/levi_01.webp	Neue Treter 🔥
dummy/levi_02.webp	Heimspiel heute!
dummy/levi_03.webp	Tor in der 89. ⚽

> **HINWEIS**
>
> Wer mit `name = 'Levi Schumacher'` sucht, bekommt auch die Fotos von levi.berlin.

d) Gib alle Kommentare und die Benutzernamen der Verfasser aus München aus.

```sql
SELECT users.username, comments.body
FROM comments
JOIN users ON comments.user_id = users.id
WHERE users.city = 'München'
```

Ergebnis: 34 Zeilen (Ausschnitt)

username	body
annalisa.f	Rezept bitte!
annalisa.f	Fernweh!!
annalisa.f	😍

e) Gib alle Kommentare aus, die unter dem Foto mit der Ostsee (`dummy/linus_seascape.webp`) stehen.

```sql
SELECT comments.body, comments.created_at
FROM comments
JOIN photos ON comments.photo_id = photos.id
WHERE photos.image = 'dummy/linus_seascape.webp'
```

Ergebnis: 4 Zeilen (Ausschnitt)

body	created_at
Hammer	2025-02-23 18:54:37
👍	2025-08-30 12:33:49
Hammer	2025-10-07 01:47:59

f) Welches Foto hat die meisten Kommentare?

```sql
SELECT photos.image, photos.description, COUNT(*) AS kommentare
FROM comments
JOIN photos ON comments.photo_id = photos.id
GROUP BY photos.id
ORDER BY kommentare DESC
LIMIT 1
```

Ergebnis: 1 Zeile

image	description	kommentare
dummy/giulia_03.webp	🎁 GEWINNSPIEL: markiere …	16

> **HINWEIS**
>
> Gewinnspiel-Post von giulia.glow (16). Wem das Foto gehört, kommt in Aufgabe 6 b).

g) Ein Kommentar taucht auffällig oft auf. Finde ihn und seinen Verfasser.

```sql
SELECT users.username, comments.body, COUNT(*) AS anzahl
FROM comments
JOIN users ON comments.user_id = users.id
GROUP BY comments.body
ORDER BY anzahl DESC
LIMIT 1
```

Ergebnis: 1 Zeile

username	body	anzahl
gewinn_jetzt_2026	Gratis Follower 💰👉 Link…	30

> **HINWEIS**
>
> „Gratis Follower 💰👉 Link in Bio“ von gewinn_jetzt_2026, 30-mal – ein Spam-Bot.

h) Die Moderation deines Netzwerks bittet um Mithilfe: Einige User verbreiten, andere seien „Echsenmenschen von der Rückseite des Mondes“. Wer hat solche Kommentare geschrieben? Gib E-Mail, Benutzername und Kommentar aus. Prüfe dein Ergebnis: Sind wirklich alle Treffer Verschwörungstheoretiker?

```sql
SELECT users.email, users.username, comments.body
FROM comments
JOIN users ON comments.user_id = users.id
WHERE comments.body LIKE '%Echse%'
```

Ergebnis: 26 Zeilen (Ausschnitt)

email	username	body
echsenjaeger@aluhut.test	mondrueckseite	Die Echsen von der Mondrü…
echsenjaeger@aluhut.test	mondrueckseite	Schaut euch die Augen vom…
echsenjaeger@aluhut.test	mondrueckseite	Wieder ein Beweis für die…

> **HINWEIS**
>
> Die Theoretiker: mondrueckseite (Ronny Pfeiffer) und echsenwache (Sven Kühn). Falle: Treffer sind auch harmlose Kommentare („Süße Echse!!“ unter Sofias Bartagame) und Antworten anderer, die @echsenwache erwähnen. Unter welchen Bildern sie stehen, kommt in Aufgabe 6 a). Wichtig für Aufgabe 11 e).

Aufgabe 6

a) Unter welchen Bildern stehen die Echsen-Kommentare aus Aufgabe 5 h)? Gib Benutzername, Kommentar und Bild aus.

```sql
SELECT users.username, comments.body, photos.image
FROM users
JOIN comments ON comments.user_id = users.id
JOIN photos ON comments.photo_id = photos.id
WHERE comments.body LIKE '%Echse%'
```

Ergebnis: 26 Zeilen (Ausschnitt)

username	body	image
mondrueckseite	Die Echsen von der Mondrü…	dummy/levi_02.webp
mondrueckseite	Schaut euch die Augen vom…	dummy/ralf_01.webp
mondrueckseite	Wieder ein Beweis für die…	dummy/henriette_01.webp

b) ★ Welches Foto hat die meisten Kommentare – und wem gehört es? *Achtung: Hier gibt es zwei verschiedene User – den Verfasser des Kommentars und den Besitzer des Fotos.*

```sql
SELECT photos.image, users.username AS besitzer, COUNT(*) AS kommentare
FROM comments
JOIN photos ON comments.photo_id = photos.id
JOIN users ON photos.user_id = users.id
GROUP BY photos.id
ORDER BY kommentare DESC
LIMIT 1
```

Ergebnis: 1 Zeile

image	besitzer	kommentare
dummy/giulia_03.webp	giulia.glow	16

> **HINWEIS**
>
> giulia.glow (16, Gewinnspiel). Wer users über comments.user_id joint, bekommt die Verfasser statt der Besitzerin.

c) Gib eine Tabelle aus, in der alle Fotos zusammen mit ihren Tags stehen.

```sql
SELECT photos.image, tags.name
FROM photos
JOIN photo_tag ON photo_tag.photo_id = photos.id
JOIN tags ON tags.id = photo_tag.tag_id
```

Ergebnis: 242 Zeilen (Ausschnitt)

image	name
dummy/paul_01.webp	gaming
dummy/paul_01.webp	setup
dummy/paul_02.webp	gaming

d) Wie viele Bilder haben den Tag „abenteuerlust“? Vergleiche dein Ergebnis mit `LIKE '%abenteuer%'`. Welche Zahl ist richtig?

BEFEHL 1

```sql
SELECT COUNT(*)
FROM photo_tag
JOIN tags ON tags.id = photo_tag.tag_id
WHERE tags.name = 'abenteuerlust'
```

Ergebnis: 1 Zeile

COUNT(*)
4

BEFEHL 2

```sql
SELECT tags.name, COUNT(*)
FROM photo_tag
JOIN tags ON tags.id = photo_tag.tag_id
WHERE tags.name LIKE '%abenteuer%'
GROUP BY tags.name
```

Ergebnis: 3 Zeilen

name	COUNT(*)
abenteuer	1
abenteuer_lust	1
abenteuerlust	4

HINWEIS

Genau: 4. Mit LIKE 6, weil es auch „abenteuer“ und „abenteuer_lust“ gibt. Je nach Frage ist beides vertretbar.

e) Welche Tags hat Alina Keller genutzt?

```sql
SELECT DISTINCT tags.name
FROM users
JOIN photos ON photos.user_id = users.id
JOIN photo_tag ON photo_tag.photo_id = photos.id
JOIN tags ON tags.id = photo_tag.tag_id
WHERE users.name = 'Alina Keller'
```

Ergebnis: 9 Zeilen (Ausschnitt)

name
ootd
fashion
herbst

HINWEIS

Vier Tabellen. Ohne DISTINCT erscheinen Tags doppelt.

f) Gib die 10 beliebtesten Fotos aus (die meisten Likes) – mit dem Benutzernamen des Besitzers.

```sql
SELECT photos.image, users.username, COUNT(*) AS likes
FROM likes
JOIN photos ON likes.photo_id = photos.id
JOIN users ON photos.user_id = users.id
GROUP BY photos.id
ORDER BY likes DESC
LIMIT 10
```

Ergebnis: 10 Zeilen (Ausschnitt)

image	username	likes
dummy/brigitte_05.webp	backmitbrigitte	59
dummy/alex_05.webp	alex.achen	19
dummy/lena_05.webp	lostwithlena	16

g) Erinnerst du dich an Erna aus Aufgabe 3 j)? Wem folgt sie? Ist ihr Enkel dabei?

```sql
SELECT star.username, star.name
FROM follows
JOIN users AS fan ON follows.follower_id = fan.id
JOIN users AS star ON follows.following_id = star.id
WHERE fan.username = 'erna.lueders'
```

Ergebnis: 9 Zeilen (Ausschnitt)

username	name
fitbyfee	Fee Lorenz
vanlife_tomasz	Tomasz Kowalski
alex.achen	Alexander Achen

> **HINWEIS**
>
> Ja: lasse.ld.

h) ★ Welche User folgen sich gegenseitig?

```sql
SELECT a.username, b.username
FROM follows AS f1
JOIN follows AS f2 ON f1.follower_id = f2.following_id AND f1.following_id = f2.follower_id
JOIN users AS a ON a.id = f1.follower_id
JOIN users AS b ON b.id = f1.following_id
WHERE a.id < b.id
```

Ergebnis: 106 Zeilen (Ausschnitt)

username	username
pixelpaul	noobqueen
pixelpaul	hugo.ranked
pixelpaul	inkandmila

> **HINWEIS**
>
> follows zweimal mit Alias. `a.id < b.id` verhindert, dass jedes Paar doppelt erscheint.

Aufgabe 7

a) Liste alle User auf, die aus Deutschland kommen – egal, ob sie „Deutschland“, „Germany“, „deutschland“ oder „DE“ angegeben haben.

```sql
SELECT username, country
FROM users
WHERE country IN ('Deutschland', 'Germany', 'deutschland', 'DE')
```

Ergebnis: 60 Zeilen (Ausschnitt)

username	country
pixelpaul	Deutschland
noobqueen	Deutschland
hugo.ranked	Deutschland

b) Gib alle User aus, die noch nie ein Foto gepostet haben. Nutze NOT IN.

```sql
SELECT username
FROM users
WHERE id NOT IN (SELECT user_id FROM photos)
```

Ergebnis: 16 Zeilen (Ausschnitt)

username
aaron183
echsenwache
fmroth

> **HINWEIS**
>
> 16 User (die Trolle, Kommentierer und leeren Konten).

c) Gib den Namen der größten Person aus – diesmal sicher richtig (vgl. Aufgabe 4 c).

```sql
SELECT name, centimeter
FROM users
WHERE centimeter = (SELECT MAX(centimeter) FROM users)
```

Ergebnis: 1 Zeile

name	centimeter
Ronny Pfeiffer	250

d) Welche Tags wurden mindestens 8-mal verwendet?

```sql
SELECT tags.name, COUNT(*) AS anzahl
FROM tags
JOIN photo_tag ON photo_tag.tag_id = tags.id
GROUP BY tags.id
HAVING COUNT(*) >= 8
ORDER BY anzahl DESC
```

Ergebnis: 6 Zeilen (Ausschnitt)

name	anzahl
gaming	15
reisen	13
fussball	11

e) Finde die 5 Fotos, die von niemandem gelikt wurden. Nutze den LEFT JOIN.

```sql
SELECT photos.image, photos.description
FROM photos
LEFT JOIN likes ON likes.photo_id = photos.id
WHERE likes.photo_id IS NULL
```

Ergebnis: 5 Zeilen (Ausschnitt)

image	description
dummy/ralf_02.webp	Plan für Samstag
dummy/ole_04.webp	*NULL*
dummy/larsole_01.webp	*NULL*

> **HINWEIS**
>
> ole_04, ralf_02, ben_01, larsole_01, jakob_01.

f) ★ Liste die Bilder, die nicht gelikt wurden, noch einmal auf – diesmal mit dem RIGHT JOIN. Was musst du gegenüber Aufgabe 7 e) ändern?

```sql
SELECT photos.image
FROM likes
RIGHT JOIN photos ON likes.photo_id = photos.id
WHERE likes.photo_id IS NULL
```

Ergebnis: 5 Zeilen (Ausschnitt)

image
dummy/ralf_02.webp
dummy/ole_04.webp
dummy/larsole_01.webp

> **HINWEIS**
>
> Die Reihenfolge der Tabellen tauschen: photos muss rechts stehen.

g) Beide Befehle sollen zählen, wie viele Fotos jeder User hat. Bei manchen Usern unterscheiden sich die Zahlen. Bei wem und warum?

BEFEHL 1

```sql
SELECT users.username, COUNT(*)
FROM users
LEFT JOIN photos ON photos.user_id = users.id
GROUP BY users.id
ORDER BY 2
```

Ergebnis: 70 Zeilen (Ausschnitt)

username	COUNT(*)
aaron183	1
realtalk_dennis	1
mondrueckseite	1

BEFEHL 2

```
SELECT users.username, COUNT(photos.id)
FROM users
LEFT JOIN photos ON photos.user_id = users.id
GROUP BY users.id
ORDER BY 2
```

Ergebnis: 70 Zeilen (Ausschnitt)

username	COUNT(photos.id)
aaron183	0
realtalk_dennis	0
mondrueckseite	0

HINWEIS

User ohne Foto: COUNT(*) zählt ihre eine Zeile mit NULL als 1, COUNT(photos.id) zählt NULL nicht und gibt 0.

h) Einer der beiden Befehle läuft nicht. Welcher, und warum? Was musst du ändern?

```
SELECT city, COUNT(*)
FROM users
GROUP BY city
HAVING COUNT(*) > 4
```

Ergebnis: 4 Zeilen (Ausschnitt)

city	COUNT(*)
Berlin	7
Kiel	11
Leipzig	5

HINWEIS

WHERE COUNT(*) > 4 ist ein Fehler: WHERE kommt vor der Gruppierung, da gibt es noch kein COUNT. Richtig: HAVING.

i) ★ Welche User haben mehr Fotos gepostet als der Durchschnitt aller User, die posten?

```
SELECT users.username, COUNT(*) AS fotos
FROM users
JOIN photos ON photos.user_id = users.id
GROUP BY users.id
HAVING COUNT(*) > (SELECT AVG(anzahl) FROM (SELECT COUNT(*) AS anzahl FROM photos GROUP BY
user_id) AS t)
```

Ergebnis: 22 Zeilen (Ausschnitt)

username	fotos
pixelpaul	6
noobqueen	5
hugo.ranked	4

HINWEIS

Unterabfrage in der Unterabfrage – nach HAVING.

Aufgabe 8

a) Gib eine Liste aus, in der alle User (Username, Name, Country) sortiert nach ihrem Namen stehen.

```
SELECT username, name, country
FROM users
ORDER BY name
```

Ergebnis: 70 Zeilen (Ausschnitt)

username	name	country
gewinn_jetzt_2026	*NULL*	*NULL*
test_account	*NULL*	*NULL*
aaron183	Aaron Krämer	Deutschland

b) Erstelle eine Liste, in der steht, wie viele User aus den jeweiligen Städten kommen – sortiert nach der Anzahl der User.

```
SELECT city, COUNT(*) AS anzahl
FROM users
GROUP BY city
ORDER BY anzahl DESC
```

Ergebnis: 34 Zeilen (Ausschnitt)

city	anzahl
Kiel	11
München	7
Berlin	7

c) Wie viele User haben eine E-Mail-Adresse deines Netzwerks und wie viele eine externe?

BEFEHL 1

```
SELECT COUNT(*)
FROM users
WHERE email LIKE '%@liky.test'
```

Ergebnis: 1 Zeile

COUNT(*)
39

BEFEHL 2

```
SELECT COUNT(*)
FROM users
WHERE email NOT LIKE '%@liky.test'
```

Ergebnis: 1 Zeile

COUNT(*)
29

> **HINWEIS**
>
> Vorsicht: Konten ohne E-Mail (NULL) zählt keiner der beiden Befehle.

d) Gib die drei jüngsten User aus.

```sql
SELECT name, birthday
FROM users
WHERE birthday IS NOT NULL
ORDER BY birthday DESC
LIMIT 3
```

Ergebnis: 3 Zeilen

name	birthday
Lotta Berg	2014-04-16
Ida Hansen	2013-08-09
Marie Schuster	2012-10-08

> **HINWEIS**
>
> Ohne `IS NOT NULL` kann je nach Datenbank NULL ganz oben stehen.

e) Welcher User hat die meisten Fotos gepostet?

```sql
SELECT users.username, COUNT(*) AS fotos
FROM users
JOIN photos ON photos.user_id = users.id
GROUP BY users.id
ORDER BY fotos DESC
LIMIT 1
```

Ergebnis: 1 Zeile

username	fotos
alina.daily	7

f) Gib eine Liste aus, in der für jeden User die Anzahl der von ihm verteilten Likes steht – auch für User ohne Like.

braucht Abschnitt 4 (Tiefer in SQL)

```sql
SELECT users.username, COUNT(likes.photo_id) AS likes
FROM users
LEFT JOIN likes ON likes.user_id = users.id
GROUP BY users.id
ORDER BY likes DESC
```

Ergebnis: 70 Zeilen (Ausschnitt)

username	likes
laura.mauer	46
stefan.b	40
marlo.eb	40

g) Welche Fotos hat hugo.ranked kommentiert, und was hat er geschrieben?

```sql
SELECT photos.image, comments.body
FROM comments
JOIN users ON comments.user_id = users.id
JOIN photos ON comments.photo_id = photos.id
WHERE users.username = 'hugo.ranked'
```

Ergebnis: 7 Zeilen (Ausschnitt)

image	body
dummy/finn_01.webp	😍
dummy/marco_04.webp	🤔
dummy/mia_04.webp	Krass 🔥

h) Gib eine Liste aus, in der alle Bildbeschreibungen stehen, die von Laura Mauer oder Marlo Eberhart gelikt wurden.

```
SELECT DISTINCT photos.description
FROM photos
JOIN likes ON likes.photo_id = photos.id
JOIN users ON likes.user_id = users.id
WHERE users.name IN ('Laura Mauer', 'Marlo Eberhart')
```

Ergebnis: 61 Zeilen (Ausschnitt)

description
Welcher ist euer Favorit?
Leg Day 🦵
Bootcamp im Park ☀

i) ★ Welcher Tag wird von den meisten verschiedenen Usern benutzt?

```
SELECT tags.name, COUNT(DISTINCT photos.user_id) AS user
FROM tags
JOIN photo_tag ON photo_tag.tag_id = tags.id
JOIN photos ON photos.id = photo_tag.photo_id
GROUP BY tags.id
ORDER BY user DESC
LIMIT 1
```

Ergebnis: 1 Zeile

name	user
reisen	9

> **HINWEIS**
>
> COUNT(DISTINCT …) zählt jeden User nur einmal.

j) Welche User haben noch nie etwas gepostet, gelikt oder kommentiert?

braucht Abschnitt 4 (Tiefer in SQL)

```
SELECT username
FROM users
WHERE id NOT IN (SELECT user_id FROM photos) AND id NOT IN (SELECT user_id FROM likes) AND id
NOT IN (SELECT user_id FROM comments)
```

Ergebnis: 4 Zeilen (Ausschnitt)

username
geloescht_4711
jule.w
max.mueller

> **HINWEIS**
>
> Die leeren Konten (max.mueller, test_account, jule.w, geloescht_4711) – und das eigene Admin-Konto.

Aufgabe 9

a) Füge eine fiktive Person zu deiner Datenbank hinzu.

```
INSERT INTO users (username, name, email, password, gender, birthday, city, country,
centimeter)
VALUES ('guenther37', 'Günther Müller', 'guenther@mail.test', '123passwort', 'male', '1987-07-
07', 'Kiel', 'Deutschland', 178)
```

→ 1 Zeile(n) geändert

b) Überprüfe, welche id diese Person bekommen hat, obwohl du sie nicht angegeben hast. Welche Werte stehen in role und created_at? Woher kommen sie? (Schau ins Diagramm.)

Vorher angenommen: `INSERT INTO users (username, name, password) VALUES ('guenther37', 'Günther Müller', '123passwort')`

```
SELECT id, role, created_at
FROM users
WHERE username = 'guenther37'
```

Ergebnis: 1 Zeile

id	role	created_at
71	user	2026-11-08 22:50:48

> **HINWEIS**
>
> id fortlaufend (AUTO_INCREMENT), role = user (DEFAULT), created_at = jetzt (DEFAULT CURRENT_TIMESTAMP).

c) Melde dich in der App mit dieser Person an. Es klappt nicht! Aber warum? Finde heraus, woran es liegt.

Vorher angenommen: `INSERT INTO users (username, name, password) VALUES ('guenther37', 'Günther Müller', '123passwort')`

BEFEHL 1

```
SELECT username, password
FROM users
WHERE username IN ('guenther37', 'pixelpaul')
```

Ergebnis: 2 Zeilen

username	password
guenther37	123passwort
pixelpaul	$2a$04$zYntaxh3Fl1bL/Vl3J…

BEFEHL 2

```
UPDATE users
SET password = PASSWORD_HASH('123passwort')
WHERE username = 'guenther37'
```

→ 1 Zeile(n) geändert

> **HINWEIS**
>
> Bei pixelpaul steht ein bcrypt-Hash ($2a$08$…), bei der neuen Person der Klartext. Die App vergleicht nur Hashes. Gespräch: Warum speichert man Passwörter nie im Klartext? Siehe Kasten.

d) Schreibe unter ein Foto deiner Wahl Kommentare im Namen von mindestens drei verschiedenen Usern. Schau in der App nach: Kann man erkennen, dass die Kommentare nicht echt sind?

```
INSERT INTO comments (user_id, photo_id, body)
VALUES (8, 1, 'Wow, wie schön!'), (15, 1, 'Bestes Foto 🔥'), (22, 1, 'Das will ich auch!')
```

→ 3 Zeile(n) geändert

> **HINWEIS**
>
> Nicht zu erkennen. Gespräch: Wer kann so etwas auf echten Plattformen? Fake-Kommentare und gekaufte Follower.

e) Gönn dir ein paar mehr Follower: Alle User aus München sollen deinem Admin-Konto folgen. Erstelle dafür keine neuen Personen.

Vorher angenommen: `INSERT INTO users (id, username, password) VALUES (71, 'admin', 'x')`

```
INSERT INTO follows (follower_id, following_id)
SELECT id, 71
FROM users
WHERE city = 'München'
```

→ 7 Zeile(n) geändert

> **HINWEIS**
>
> Die eigene id zuerst mit SELECT herausfinden (hier 71). Folgt schon jemand, bricht der Befehl mit Fehler ab (zusammengesetzter Primärschlüssel).

f) Lass 20 User ein Foto deiner Wahl mit 😂 (laugh) liken. Was passiert, wenn einer von ihnen das Foto schon gelikt hat? Lies die Fehlermeldung und erkläre sie mit dem Diagramm.

braucht Abschnitt 4 (Tiefer in SQL)

```
INSERT INTO likes (user_id, photo_id, kind)
SELECT id, 5, 'laugh'
FROM users
WHERE id NOT IN (SELECT user_id FROM likes WHERE photo_id = 5)
LIMIT 20
```

→ 20 Zeile(n) geändert

> **HINWEIS**
>
> Ohne NOT IN: Fehler wegen des Primärschlüssels (user_id, photo_id) – jede Kombination darf es nur einmal geben.

Aufgabe 10

a) Versuche, das Geschlecht aller User aus Dresden auf „keine Angabe“ zu setzen. Lies die Fehlermeldung: Welche Werte sind erlaubt? Wo im Diagramm steht das? Warum ist so eine Einschränkung sinnvoll?

```
UPDATE users
SET gender = 'keine Angabe'
WHERE city = 'Dresden'
```

→ Fehlermeldung: *Dieser Wert ist nicht erlaubt.*

> **HINWEIS**
>
> Fehler: gender ist ein ENUM mit male, female, diverse. „Keine Angabe“ wäre NULL.

b) Suche dir 10 Bilder von anderen Usern deiner Datenbank aus und mache sie zu deinen Bildern. Wie sieht dein Profil danach aus?

Vorher angenommen: `INSERT INTO users (id, username, password) VALUES (71, 'admin', 'x')`

```sql
UPDATE photos
SET user_id = 71
WHERE id IN (3, 17, 24, 38, 51, 66, 72, 90, 104, 120)
```

→ 10 Zeile(n) geändert

> **HINWEIS**
>
> Likes und Kommentare wandern mit – sie hängen am Foto, nicht am User.

c) ★ Lenke alle Likes auf eines deiner Bilder um. Warum klappt das nicht? *Tipp: Schau dir den Primärschlüssel von likes an.* Schaffst du es, wenigstens die Likes umzulenken, die möglich sind?

braucht Abschnitt 4 (Tiefer in SQL)

BEFEHL 1

```sql
UPDATE likes
SET photo_id = 5
```

→ Fehlermeldung: *Diesen Wert in »likes.user_id, likes.photo_id« gibt es schon.*

BEFEHL 2

```sql
UPDATE likes
SET photo_id = 5
WHERE user_id NOT IN (SELECT user_id FROM likes WHERE photo_id = 5) AND (user_id, photo_id) IN
(SELECT user_id, MIN(photo_id) FROM likes GROUP BY user_id)
```

→ 53 Zeile(n) geändert

> **HINWEIS**
>
> Der erste Befehl scheitert: Jeder User hat viele Likes, nach dem UPDATE gäbe es (user_id, 5) mehrfach. Die zweite Lösung nimmt je User nur ein Like und lässt User aus, die Foto 5 schon liken.

d) Ersetze die Körpergröße aller User durch `FLOOR(RAND() * 50) + 150`. Was erzeugt dieser Befehl? Führe ihn zweimal aus und vergleiche.

```sql
UPDATE users
SET centimeter = FLOOR(RAND() * 50) + 150
```

→ 70 Zeile(n) geändert

> **HINWEIS**
>
> Zufällige Größen zwischen 150 und 199 cm – bei jedem Ausführen andere.

e) User, die ihre Boote fotografiert haben, möchten ihren Wohnort durch „auf hoher See“ ersetzen. Zwei Admins schlagen verschiedene Befehle vor (rechts). Welcher erfüllt den Wunsch besser? Prüfe es erst mit SELECT.

braucht Abschnitt 4 (Tiefer in SQL)

BEFEHL 1

```
SELECT DISTINCT users.username
FROM users
JOIN photos ON photos.user_id = users.id
JOIN photo_tag ON photo_tag.photo_id = photos.id
JOIN tags ON tags.id = photo_tag.tag_id
WHERE tags.name = 'boot'
```

Ergebnis: 3 Zeilen

username
finn.kielwasser
werner.angelt
mikkel.s

BEFEHL 2

```
UPDATE users
SET city = 'auf hoher See'
WHERE id IN (SELECT photos.user_id FROM photos JOIN photo_tag ON photo_tag.photo_id = photos.id
JOIN tags ON tags.id = photo_tag.tag_id WHERE tags.name = 'boot')
```

→ 3 Zeile(n) geändert

HINWEIS

Über den Tag „boot“: finn.kielwasser, werner.angelt, mikkel.s. Die Textsuche trifft auch fitbyfee („Bootcamp im Park“).

f) hugo.ranked hat sein Passwort vergessen. Setze es neu und melde dich danach als Hugo an.

```
UPDATE users
SET password = PASSWORD_HASH('neu123')
WHERE username = 'hugo.ranked'
```

→ 1 Zeile(n) geändert

g) Der Sicherheitsbeauftragte vermutet, dass manche User sehr schwache Passwörter haben. Finde mit `PASSWORD_VERIFY` heraus, wer „123456“ benutzt. Probiere auch „passwort“ und „hallo123“. Erkläre: Warum kann man den Hash nicht zurückrechnen, die Passwörter aber trotzdem finden?

BEFEHL 1

```
SELECT username
FROM users
WHERE PASSWORD_VERIFY('123456', password)
```

Ergebnis: 2 Zeilen

username
coach_ralf
werner.angelt

BEFEHL 2

```
SELECT username
FROM users
WHERE PASSWORD_VERIFY('passwort', password) OR PASSWORD_VERIFY('hallo123', password)
```

Ergebnis: 2 Zeilen

username
hugo.ranked
ida.hsn

> **HINWEIS**
>
> 123456: coach_ralf, werner.angelt; passwort: hugo.ranked; hallo123: ida.hsn. Man rät Passwörter und vergleicht die Hashes – bei häufigen Passwörtern klappt das schnell. Darum: lange, eigene Passwörter.

Aufgabe 11

a) Lösche alle Lisas aus deiner Datenbank. Vergleiche vorher `name LIKE 'Lisa%'` und `name LIKE '%Lisa%'`. Wie viele Lisas gibt es wirklich?

BEFEHL 1

```sql
SELECT name
FROM users
WHERE name LIKE '%Lisa%'
```

Ergebnis: 4 Zeilen (Ausschnitt)

name
Lisa Meier
Lisa-Marie Kraft
Elisa Brandt

BEFEHL 2

```sql
DELETE
FROM users
WHERE name LIKE 'Lisa%'
```

→ 2 Zeile(n) geändert

> **HINWEIS**
>
> %Lisa% trifft auch Elisa Brandt und Annalisa Ferri. Lisas: Lisa Meier und Lisa-Marie Kraft.

b) Lösche alle Bilder von Alexander Achen. Was passiert mit den Likes und Kommentaren an seinen Bildern?

```sql
DELETE
FROM photos
WHERE user_id = (SELECT id FROM users WHERE name = 'Alexander Achen')
```

→ 5 Zeile(n) geändert

> **HINWEIS**
>
> CASCADE: Likes, Kommentare und Tag-Zuordnungen seiner Fotos verschwinden mit.

c) ★ Lösche alles, was mit Henriette Zimmerman in Verbindung steht, ihren Account jedoch nicht: alle Fotos, alle Kommentare, alle Follower, alle Abos und ihr Profilbild. *Tipp: Das Profilbild ist kein eigenes Objekt.*

BEFEHL 1

```sql
DELETE
FROM photos
WHERE user_id = 19
```

→ 5 Zeile(n) geändert

BEFEHL 2

```
DELETE
FROM comments
WHERE user_id = 19
```

→ 3 Zeile(n) geändert

BEFEHL 3

```
DELETE
FROM follows
WHERE follower_id = 19 OR following_id = 19
```

→ 20 Zeile(n) geändert

BEFEHL 4

```
DELETE
FROM likes
WHERE user_id = 19
```

→ 14 Zeile(n) geändert

BEFEHL 5

```
UPDATE users
SET avatar = NULL
WHERE id = 19
```

→ 1 Zeile(n) geändert

HINWEIS

id von wuffi_und_ich zuerst mit SELECT ermitteln (hier 19). Das Profilbild ist ein Attribut → UPDATE auf NULL, kein DELETE.

d) Lösche alle Personen unter 18. Wie viele Personen waren das?

```
DELETE
FROM users
WHERE FLOOR(DATEDIFF(NOW(), birthday) / 365.25) < 18
```

→ 23 Zeile(n) geändert

HINWEIS

23 Personen. Achtung: Danach fehlen in Kapitel 7 u. a. tim.richter und ida.hsn – Lehrkraft füllt die Daten vorher neu.

e) Verschwörungstheorien über Echsenmenschen werden in deinem Netzwerk nicht geduldet. Lösche alle Kommentare der Echsen-Theoretiker – aber nur diese! Sofias harmlose Bartagame und die Antworten anderer User sollen bleiben. Vergleiche dazu die beiden Befehle rechts mit SELECT (vgl. Aufgabe 5 h).

braucht Abschnitt 4 (Tiefer in SQL)

BEFEHL 1

```
SELECT COUNT(*)
FROM comments
WHERE body LIKE '%Echse%'
```

Ergebnis: 1 Zeile

COUNT(*)
26

BEFEHL 2

```
DELETE
FROM comments
WHERE body LIKE '%Echse%' AND user_id IN (SELECT id FROM users WHERE username IN
('mondrueckseite', 'echsenwache'))
```

→ 20 Zeile(n) geändert

> **HINWEIS**
>
> Nur nach Text zu filtern löscht zu viel (26 Treffer), nur nach User löscht auch Kommentare ohne Echsen. Am besten beides kombinieren – und vorher die Trolle per SELECT bestimmen.

f) Lösche den User aaron183. Zähle vorher und nachher alle Kommentare. Warum fehlen danach Kommentare, obwohl du in comments gar nichts gelöscht hast?

BEFEHL 1

```
SELECT COUNT(*)
FROM comments
```

Ergebnis: 1 Zeile

COUNT(*)
477

BEFEHL 2

```
DELETE
FROM users
WHERE username = 'aaron183'
```

→ 1 Zeile(n) geändert

BEFEHL 3

```
SELECT COUNT(*)
FROM comments
```

Ergebnis: 1 Zeile

COUNT(*)
459

> **HINWEIS**
>
> ON DELETE CASCADE: Mit dem User verschwinden seine 18 Kommentare, Likes und Follows.

Aufgabe 12

Offene Aufgabe – jede und jeder findet etwas anderes. Beispiele für mögliche Befehle:

> **HINWEIS**
>
> Im Klassen-Netz braucht man Admin-Rechte (`UPDATE users SET role = 'admin' WHERE …`). Abschluss: „Was weiß das Netzwerk über mich?“ auslösen.

a) Beliebteste Desktop-Browser

```sql
SELECT browser, COUNT(*) AS sessions
FROM sessions
WHERE device_type = 'desktop'
GROUP BY browser
ORDER BY sessions DESC
```

Ergebnis: 4 Zeilen (Ausschnitt)

browser	sessions
Firefox 144	84
Edge 141	51
Safari 18	43

b) Gleiche IP = gleicher Ort

```sql
SELECT ip, GROUP_CONCAT(DISTINCT users.username) AS user
FROM sessions
JOIN users ON sessions.user_id = users.id
GROUP BY ip
HAVING COUNT(DISTINCT sessions.user_id) > 1
```

Ergebnis: 2 Zeilen

ip	user
203.0.113.116	lasse.ld,larsole
203.0.113.158	pixelpaul,leila.hd

> **HINWEIS**
>
> lasse.ld und larsole teilen eine Heim-IP. Im Klassen-Netz haben alle die Schul-IP.

c) Viele IPs (VPN?)

```sql
SELECT users.username, COUNT(DISTINCT sessions.ip) AS ips
FROM sessions
JOIN users ON sessions.user_id = users.id
GROUP BY users.id
HAVING COUNT(DISTINCT sessions.ip) > 5
```

Ergebnis: 2 Zeilen

username	ips
vanlife_tomasz	9
gewinn_jetzt_2026	14

> **HINWEIS**
>
> vanlife_tomasz (9) und der Bot gewinn_jetzt_2026 (14).

d) Nachts online

```sql
SELECT DISTINCT users.username, FLOOR(DATEDIFF(NOW(), birthday) / 365.25) AS jahre
FROM sessions
JOIN users ON sessions.user_id = users.id
WHERE HOUR(sessions.started_at) BETWEEN 1 AND 3
```

Ergebnis: 3 Zeilen

username	jahre
ida.hsn	13
tim.richter	14
gewinn_jetzt_2026	*NULL*

> **HINWEIS**
>
> u. a. tim.richter (14) und ida.hsn (13) – passt zur Werbung „Nachtschicht Snacks“.

e) Verbindung und Hell/Dunkel

```sql
SELECT connection, dark_mode, COUNT(*)
FROM sessions
GROUP BY connection, dark_mode
```

Ergebnis: 8 Zeilen (Ausschnitt)

connection	dark_mode	COUNT(*)
4g	0	48
4g	1	89
5g	0	54

f) Erlaubnisse

```sql
SELECT allow_location, COUNT(*)
FROM sessions
GROUP BY allow_location
```

Ergebnis: 2 Zeilen

allow_location	COUNT(*)
0	269
1	721

g) Interessen über die Verweildauer (tim.richter)

```sql
SELECT tags.name, AVG(views.duration_ms) / 1000 AS sekunden
FROM views
JOIN users ON views.user_id = users.id
JOIN photo_tag ON photo_tag.photo_id = views.photo_id
JOIN tags ON tags.id = photo_tag.tag_id
WHERE users.username = 'tim.richter'
GROUP BY tags.id
ORDER BY sekunden DESC
LIMIT 5
```

Ergebnis: 5 Zeilen (Ausschnitt)

name	sekunden
retro	19.03
stream	18.28
setup	17.37

> **HINWEIS**
>
> Gaming-Tags weit vorn, obwohl er kaum liked.

Aufgabe 13

a) Schau dir die Kampagnen in deinem Netzwerk an (`SELECT * FROM ads` oder Reiter **Werbung**). Wer sieht sie und warum? Welche findest du problematisch?

```sql
SELECT name, kind, target_sql, content_sql
FROM ads
```

Ergebnis: 9 Zeilen (Ausschnitt)

name	kind	target_sql	content_sql
VOLTA Energy	feed	SELECT l.user_id FROM lik…	*NULL*
Nachtschicht Snacks	feed	SELECT COUNT(*) > 0 FROM …	*NULL*
SpeedPhone X	feed	SELECT COUNT(*) > 0 FROM …	*NULL*

> **HINWEIS**
>
> Problematisch z. B. „Nachtschicht Snacks“ (nachts online – oft Minderjährige) und „Wahrheit.tv“ (Flacherde-Posts für Leute, die so etwas liken).

b) Denkt euch einen Werbekunden aus (Produkt, Firma, Partei oder Organisation) und überlegt, welche User-Gruppen ihr wie ansprechen wollt.

Offene Aufgabe – keine feste Lösung.

c) Entwickelt **drei Werbungen** für euren Kunden – eine **Feed-Werbung**, ein **Foto-Label** und einen **beworbenen Post**, jede mit eigener Zielgruppe per SQL. Testet sie in eurem Netzwerk (Probelauf, Vorschau) und schaltet sie dann im **Klassen-Netz**: Möglichst jede Person im Kurs soll eine passende Werbung bekommen.

Offene Aufgabe – keine feste Lösung.

> **HINWEIS**
>
> Im Klassen-Netz brauchen die Gruppen Admin-Rechte und es gibt dort keine Dummy-User – die Zielgruppen müssen auf die echten Daten des Kurses passen.